

Doc. no.	P1056R1
Revises:	P1056R0
Date:	2018-10-07
Audience:	LWG
Reply to:	Lewis Baker < lewissbaker@gmail.com >, Gor Nishanov < gorn@microsoft.com >

Add lazy coroutine (coroutine task) type

Overview

The coroutines TS has introduced a language capability that allows functions to be suspended and later resumed. One of the key applications for this new feature is to make it easier to write asynchronous code. However, the coroutines TS itself does not include any concrete coroutine types that directly support writing asynchronous code.

This paper proposes adding a new type, `std::lazy<T>`, to the standard library to enable creation and composition of coroutines representing asynchronous computation.

[Note: SG1 would like to keep the name `std::task` for something else. SG1 requested to pick a different name. LEWG picked `lazy`. endnote --]

```
#include <experimental/lazy>
#include <string>

struct record
{
    int id;
    std::string name;
    std::string description;
};

std::lazy<record> load_record(int id);
std::lazy<> save_record(record r);

std::lazy<void> modify_record()
{
    record r = co_await load_record(123);
    r.description = "Look, ma, no blocking!";
    co_await save_record(std::move(r));
}
```

The interface of lazy is intentionally minimalistic and designed for efficiency. In fact, the only operation you can do with the lazy is to await on it.

```
template <typename T>
class [[nodiscard]] lazy {
public:
    lazy(lazy&& rhs) noexcept;
    ~lazy();
    unspecified operator co_await(); // exposition only
};
```

While such small interface may seem unusual at first, subsequent sections will clarify the rationale for this design.

Why not use futures with `future.then`?

The `std::future` type is inherently inefficient and cannot be used for efficient composition of asynchronous operations. The unavoidable overhead of futures is due to:

- allocation/deallocation of the shared state object
- atomic increment/decrement for managing the lifetime of the shared state object
- synchronization between setting of the result and getting the result
- (with `.then`) scheduling overhead of starting execution of subscribers to `.then`

Consider the following example:

```
lazy<int> coro() {
    int result = 0;
    while (int v = co_await async_read())
        result += v;
    co_return result;
}
```

where `async_read()` is some asynchronous operation that takes, say 4ns, to perform. We would like to factor out the logic into two coroutines:

```
lazy<int> subtask() { co_return co_await async_read(); }

lazy<int> coro_refactored() {
    int result = 0;
    while (int v = co_await subtask())
        result += v;
    co_return result;
}
```

Though, in this example, breaking a single `co_await` into its own function may seem silly, it is a technique that allows us to measure the overhead of composition of tasks. With proposed lazy, our per operation cost grew from 4ns to 6ns and did not incur any heap allocations. Moreover, this overhead of 2ns is not inherent to the lazy and we can anticipate that with improved coroutine optimization technologies we will be able to drive the overhead to be close to zero.

To estimate the cost of composition with `std::future`, we used the following code:

```
int fut_test() {
    int count = 1'000'000;
    int result = 0;

    while (count > 0) {
        promise p;
        auto fut = p.get_future();
        p.set_value(count--);
        result += fut.get();
    }
    return result;
}
```

As measured on the same system (Linux, clang 6.0, libc++), we get **133ns** per operation! Here is the visual illustration.

```
    op cost: ****
    lazy overhead: **
    future overhead: *****
```

Being able to break apart bigger functions into a set of smaller ones and being able to compose software by putting together small pieces is fundamental requirement for a good software engineering since the 60s. The overhead of `std::future` and types similar in behavior makes them unsuitable coroutine type.

Removing future overhead: Part 1. Memory Allocation

Consider the only operation that is available on a lazy, namely, awaiting on it.

```
lazy<X> g();
lazy<Y> f() { .... X x = co_await g(); ... }
```

The caller coroutine `f` owns the lazy object for `g` that is created and destroyed at the end of the full expression containing `co_await`. This allows the compiler to determine the lifetime of the coroutine and apply Heap Allocation Elision Optimization [P0981R0] that eliminates allocation of a coroutines state by making it a temporary variable in its caller.

Removing future overhead: Part 2. Reference counting

The coroutine state is not shared. The lazy type only allows moving pointer to a coroutine from one lazy object to another. Lifetime of a coroutine is linked to its lazy object and lazy object destructors destroys the coroutine, thus, no reference counting is required.

In a later section about Cancellation we will cover the implications of this design decision.

Removing future overhead: Part 3. Set/Get synchronization

The lazy coroutine always starts suspended. This allows not only to avoid synchronization when attaching a continuation, but also enables solving via composition how and where coroutine needs to get executed and allows to implement advanced execution strategies like continuation stealing.

Consider this example:

```
lazy<int> fib(int n) {
    if (n < 2)
        return n;
    auto xx = co_await cilk_spawn(fib(n-1)); // continuation stealing
    auto yy = fib(n-2);
    auto [x,y] = co_await cilk_sync(xx,yy);
    co_return x + y;
}
```

In here, `fib(n-1)` returns a lazy in a suspended state. Awaiting on `cilk_spawn` adapter, queues the execution of the rest of the function to a threadpool, and then resumes `f(n-1)`. Prior, this style of code was pioneered by Intel's cilk compiler and now, C++ coroutines and proposed lazy type allows to solve the problem in a similar expressive style. (Note that we in no way advocating computing fibonacci sequence in this way, however, this seems to be a popular example demonstrating the benefits of continuation stealing and we are following the trend. Also we only sketched the abstraction required to implement cilk like scheduling, there may be even prettier way).

Removing future overhead: Part 4. Scheduling overhead

Consider the following code fragment:

```
int result = 0;
while (int v = co_await async_read())
    result += v;
```

Let's say that `async_read` returns a future. That future cannot resume directly the coroutine that is awaiting on it as it will, in effect, transform the loop into unbounded recursion.

On the other hand, coroutines have built-in support for symmetric coroutine to coroutine transfer [p0913r0]. Since lazy object can only be created by a coroutine and the only way to get the result from a coroutine is by awaiting on it from another coroutine, the transfer of control from completing coroutine to awaiting coroutine is done in symmetric fashion, thus eliminating the need for extra scheduling interactions.

Destruction and cancellation

Note that the lazy type unconditionally destroys the coroutine in its destructor. It is safe to do, only if the coroutine has finished execution (at the final suspend point) or it is in a suspended state (waiting for some operation) to complete, but, somehow, we know that the coroutine will never be resumed by the entity which was supposed to resume the coroutine on behalf of the operation that coroutine is awaiting upon. That is only possible if the underlying asynchronous facility support cancellation.

We strongly believe that support for cancellation is a required facility for writing asynchronous code and we struggled for awhile trying to decide what is the source of the cancellation, whether it is the lazy, that must initiate cancellation (and therefore every await in every coroutine needs to understand how to cancel a particular operation it is being awaited upon) or every async operation is tied to a particular lifetime and cancellation domain and operations are cancelled in bulk by cancellation of the entire cancellation domain [P0399R0].

We experimented with both approaches and reached the conclusion that not performing cancellation from the lazy, but, pushing it to the cancellation domain leads to more efficient implementation and is a simpler model for the users.

Why no move assignment?

This is rather unorthodox decision and even authors of the paper did not completely agree on this topic. However, going with more restrictive model initially allows us to discover if the insight that lead to this decision was wrong. Initial design of the lazy, included move assignment, default constructor and swap. We removed them for two reasons.

First, when observing how lazy was used, we noticed that whenever, a variable-size container of tasks was created, we later realized that it was a suboptimal choice and a better solution did not require a container of tasks.

Second: move-assignment of a lazy is a ticking bomb. To make it safe, we would need to introduce per lazy cancellation of associated coroutines and it is a very heavy-weight solution.

At the moment we do not offer a move assignment, default constructor and swap. If good use cases, for which there are no better ways to solve the same problem are discovered, we can add them.

Interaction with allocators

The implementation of coroutine bindings for lazy is required to treat the case where first parameter to a coroutine is of type allocator_arg_t. If that is the case, the coroutine needs to have at least two arguments and the second one shall satisfy the Allocator requirements and if dynamic allocation required to store the coroutine state, implementation should use provided allocator to allocate and deallocate the coroutine state. Examples:

```
lazy<int> f(int, float); // uses default allocator if needed

lazy<int> f(allocator_arg_t, pmr::polymorphic_allocator<> a); // uses a to allocate, if needed

template <typename Alloc>
lazy<int> f(allocator_arg_t, Alloc a); // uses allocator a to allocate. if needed
```

Interaction with executors

Since coroutine starts suspended, it is upto the user to decide how it needs to get executed and where continuation needs to be scheduled.

Case 1: Starts in the current thread, resumes in the thread that triggered the completion of f().

```
co_await f();
```

Case 2: Starts in the current thread, resumes on executor ex.

```
co_await f().via(e);
```

[Note -- Member function via is not part of the lazy at the moment. We envision this as one possible way of how to express the desire to execute continuation in a different executor. For more details see P0975R0 that explores impact of coroutines on various existing proposals in flight -- endnote]

Case 3: Starts by an executor ex, resumes in the thread that triggered the completion of f().

```
co_await spawn(ex, f());
```

Case 4: Starts by an executor ex1, resumes on executor ex2.

```
co_await spawn(ex1, f()).via(ex2);
```

The last case is only needed if f() cannot start executing in the current thread for some reason. We expect that this will not be a common case. Usually, when a coroutine has unusual requirements on where it needs to be executed it can be encoded directly in f without forcing the callers of f, to do extra work. Typically, in this manner:

```
lazy<T> f() {
    co_await make_sure_I_am_on_the_right_thread();
    ...
}
```

But what about main?

As we mentioned in the beginning, the only operation that one can do on a lazy is to await on it (as if by `co\_await` operator). Using an *await-expression* in a function turns it into a coroutine. But, this cannot go on forever, at some point, we have to interact with coroutine from a function that is not a coroutine itself, main, for example. What to do?

There could be several functions that can bridge the gap between synchronous and asynchronous world. For example:

```
template <typename T> T sync_await(lazy<T>);
```

This function starts the lazy execution in the current thread, and, if it gets suspended, it blocks until the result is available. To simplify the signature, we show sync_await only taking objects of lazy type. This function can be written generically to handle arbitrary awaitables.

Another function could be a variant of std::async that launches execution of a lazy on a thread pool and returns a future representing the result of the computation.

```
template <typename T> T async(lazy<T>);
```

One would use this version of `std::async` if blocking behavior of `sync_await` is undesirable.

Conclusion

A version of proposed type has been used in shipping software that runs on hundreds of million devices in consumer hands. Also, a similar type has been implemented by one of the authors of this paper in most extensive coroutine abstraction library (CppCoro). This proposed type is minimal and efficient and can be used to build higher level abstraction by composition.

References

[P0981R0]: [Halo: Coroutine Heap Allocation eLision Optimization](#)
[P0913R0]: [Symmetric coroutine control transfer](#)
[P0975R0]: [Impact of coroutines on current and upcoming library facilities](#)
[CppCoro]: [A library of C++ coroutine abstractions for the coroutines TS](#)
[P0399R0]: [Networking TS & Threadpools](#)

Wording

21.11 Coroutine support library [support.coroutine]

Add the following concept definitions to synopsis of header `<experimental/coroutine>`

```
// 21.11.6 Awaitable concepts
template <typename A>
bool concept SimpleAwaitable = see below;

template <typename A>
bool concept Awaitable = see below;
```

21.11.6 Awaitable concepts [support.awaitable.simple]

1. The `Awaitable` and `SimpleAwaitable` concepts specify the requirements on a type that is usable in an *await-expression* (8.3.8).

```
template <typename T>
concept __HasMemberOperatorCoawait = exposition only
requires(T a) {
    { a.operator co_await() } -> SimpleAwaitable;
    { std::move(a).operator co_await() } -> SimpleAwaitable;
};

template <typename T>
concept __HasNonMemberOperatorCoawait = exposition only
requires(T a) {
    { operator co_await(a) } -> SimpleAwaitable;
    { operator co_await(std::move(a)) } -> SimpleAwaitable;
};

template <typename A>
concept SimpleAwaitable = requires(A a, coroutine_handle<A> h) {
    { a.await_ready() } -> bool;
    a.await_resume();
    a.await_suspend(h);
};

template <typename A>
concept Awaitable = __HasMemberOperatorCoawait<A>
|| __HasNonMemberOperatorCoawait<A> || SimpleAwaitable<A>;
```

2. If the type of an expression *E* satisfies the `Awaitable` concept then the term *simple awaitable of E* refers to an object satisfying the `SimpleAwaitable` concept that is either the result of evaluation of expression *E* itself or the result of an application (if available) of operator `co_await` to *E*.

XX.1 Coroutines tasks [coroutine.lazy]

XX.1.1 Overview [coroutine.lazy.overview]

1. This subclause describes components that a C++ program can use to create coroutines representing asynchronous computations.

XX.1.2 Header `<experimental/lazy>` synopsis [coroutine.lazy.syn]

```
namespace std::experimental {
    inline namespace coroutines_v1 {

        template<typename T = void> class lazy;
        template<typename T> class lazy<T&>;
        template<> class lazy<void>;

    } // namespace coroutines_v1
} // namespace std::experimental
```

XX.1.3 Class template `lazy` [coroutine.lazy.type]

```
template <typename T>
class [[nodiscard]] lazy {
public:
    lazy(lazy&& rhs) noexcept;
```

```

~lazy();
unspecified operator co_await() &; // exposition only
unspecified operator co_await() &&; // exposition only
};

```

1. The class template `lazy` defines a type for a *coroutine lazy object* that can be associated with a coroutine which return type is `lazy<T>` for some type `T`. In this subclause, we will refer to such a coroutine as a *lazy coroutine* and to type `T` as the *eventual type* of a coroutine.
2. The implementation shall provide specializations of `coroutine_traits` as required to implement the following behavior:
 - I. A call to a lazy coroutine `f` shall return a lazy object `t` associated with that coroutine. The called coroutine shall be suspended at the initial suspend point (11.4.4). Such a lazy object is considered to be in the *armed* state.
 - II. The type of a lazy object shall satisfy the `Awaitable` concept and awaiting on a lazy object in the armed state as if by `co_await t` (8.3.8) shall register the awaiting coroutine `a` with the lazy object `t` and resume the coroutine `f`. At this point `t` is considered to be in a *launched* state. Awaiting on a lazy object that is not in the armed state has undefined behavior.
 - III. Let `sa` be a simple awaitable of `t` (21.11.6). If the *compound-statement* of the *function-body* of the coroutine `f` completes with an unhandled exception, the awaiting coroutine `a` is resumed and an expression `sa.await_resume()` shall rethrow that exception.
 - IV. If the eventual type of a coroutine `f` is void and the coroutine completes due to an execution of a *coroutine return statement* (9.6.3), or flowing off the end of a coroutine, the awaiting coroutine `a` is resumed and an expression `sa.await_resume()` shall evaluate to void.
 - V. If the eventual type of a coroutine `f` is a non-void and the coroutine completes due to an execution of a *coroutine return statement* (9.6.3), the operand of the *coroutine return statement* shall be stored in the coroutine state and the awaiting coroutine `a` is resumed and an expression `sa.await_resume()` shall return the stored value if `t` is an rvalue or reference to a stored value, otherwise.
 - VI. If in the definition of the coroutine `g`, the first parameter has type `allocator_arg_t`, then the coroutine shall have at least two arguments and the type of the second parameter shall satisfy the `Allocator` requirements (Table 31) and if dynamic allocation is required to store the coroutine state (11.4.4), implementation shall use the provided allocator to allocate and deallocate the coroutine state.
 - VII. If a *yield-expression* (8.20) occurs in the suspension context of the lazy coroutine, the program is ill-formed.

XX.1.3.1 constructor/destructor [`coroutine.lazy.type.ctor.dtor`]

```
lazy(lazy&& rhs) noexcept;
```

1. *Effects*: Move constructs a lazy object that refers to the coroutine that was originally referred to by `rhs` (if any).
2. *Postcondition*: `rhs` shall not refer to any coroutine.

```
~lazy();
```

3. *Requires*: The coroutine referred to by the lazy object (if any) must be suspended.
4. *Effects*: The coroutine (if any) referred to by the lazy object is destroyed.